A hybrid DistilBERT-MLP framework to detect phishing links

Aryan Gahlaut
Dept. of Computer Science and engineering
Delhi Technological University (DTU)
New Delhi, India
aryangahlaut_co22a3_15@dtu.ac.in

Prabhanshu Tomer
Dept. of Computer Science and engineering
Delhi Technological University (DTU)
New Delhi, India
prabhanshutomer_co22a6_01@dtu.ac.in

Aditi Zear
Dept. of Computer Science and engineering
Delhi Technological University (DTU)
New Delhi, India
aditizear@dtu.ac.in

Ashish Raj Singh
Dept. of Computer Science and engineering
Delhi Technological University (DTU)
New Delhi, India
ashishrajsingh_co22a3_22@dtu.ac.in

***Abstract*—Phishing attacks are one of the most common cybersecurity related issues that cause severe financial damage via malicious URLs and mails. Phishing attacks via malicious URLs costs billions of dollars in damage and risk sensitive information. Existing techniques like simple ML models are sometimes incapable of generalizing well to obfuscated phishing tactics and require smarter approaches. Deep learning has become a popular technique to combat the threat of phishing attacks by using various types of neural networks[14][15]. For this purpose, we present a hybrid architecture of deep learning that involves the usage of a custom Multi layer perceptron (MLP) along with a DistilBERT model to detect phishing links. The proposed model's architecture utilizes DistilBERT's contextual representation of URLs and multi layer perceptron's ability to discriminate between feature vectors. We prepare a diverse and large dataset of 7,85,141 URLs which includes both phishing and legitimate labels. The output tensors from DistilBERT model are fed to MLP as inputs and the MLP's output tensors are then concatenated with the transformer's output tensors in feature level late fusion and then passed to the final classification layer to get the final output label. Experiments and testing shows that the proposed model significantly outperforms other ML models based on evaluation metrics. The accuracy score of 99.246% was achieved by the model in 3 epochs, while its precision, recall, and F1-score are 0.9925, respectively. These results prove that the combination of lightweight transformers and artificial neural networks is suitable for phishing attacks prevention.**

Keywords*— *Phishing link detection, cybersecurity, Multi layer perceptron, Hybrid Learning, Decision tree, Random Forest, Logistic regression, Data pre-processing, Feature representation, Transformer, Label classification.

INTRODUCTION

Today , phishing attacks usually involve links that trick people into giving away secret information like login details, bank account numbers or personal stuff. As more and more people are using the internet nowadays, the threat of exposing sensitive information is also increasing. Reports say that phishing is still a cause of security problems causing billions of dollars in damage worldwide each year [1] [2]. Old ways of detecting phishing attacks mostly rely on lists of known links and simple machine learning methods. While these lists are good at stopping known threats, they cannot catch hidden phishing links. On the other hand, simple machine learning models need people to manually create special features as URL representations via simple techniques, which requires expertise and may not work well against new attack methods [3] [4]. These problems show that we need more flexible ways to detect phishing attacks. In the last few years new hybrid ML methods have shown promise in solving these problems by automatically learning from raw data. Some models, like DistilBERT are very good at understanding patterns and relationships in text like the text in links, which makes them useful for detecting phishing attacks [5]. However to use these models well we need to prepare the data and combine them with other methods to get the best results.

In this paper, we suggest a way to detect phishing attacks that combines a DistilBERT model with a multi layer perceptron (MLP) with careful data preparation process. The data preparation step focuses on cleaning and organizing link structures to keep information consistent and remove unnecessary parts. The combined model can understand both the context and structure of links, which is helpful in detecting phishing attacks. This work focuses on phishing attacks and approaches to find them.. We will use phishing detection methods to stop phishing attacks. This proposed approach is tested on a large-scale dataset consisting of 7,85,141 URLs which comprises two different datasets, ensuring robustness and generalization across diverse phishing patterns.

The key contribution includes :

- *Hybrid architecture* - developed a dual path framework combining DistilBERT encoder transformer and specialized multi layer perceptron for improved contextual and non linear feature extraction
- *Late feature level fusion* - Output features from both the DistilBERT and MLP are fused together via concatenation.
- *Performance* - Demonstrating very high accuracy , recall , precision and f1 score of 99.25% , 0.9925, 0.9925 and 0.9925 respectively outperforming existing approaches.

RELATED WORKS

A. Traditional method - List based techniques

These list based approaches work by employing lists like whitelists and blacklists for identifying phishing URLs. List based approaches are popular due to their simplicity and very low computation cost. Examples include systems like Google Safe Browsing and PhishTank. List based approaches are often combined with heuristic based techniques or machine learning techniques to improve effectiveness in phishing detection.

From the results of recent research ,we note that while list-based approaches detect existing phishing URLs they do not provide a mechanism for detecting zero-day attacks. As mentioned earlier, most surveys pointed out that blacklisting has failed to respond to the rapidly changing characteristics of phishing where new malicious URLs are continually being generated to avoid detection [6]. Likewise, static signatures are becoming useless in combating advanced phishing campaigns using domain generation and obfuscation [7].

B. Heuristic based approaches

Heuristic based approaches look at known rules and characteristics of a phishing URL such as abnormally long URLs, the presence of special characters, the age of a domain and suspect HTML and JavaScript codes. Jayaprakash et al.[7] have designed a heuristic based framework coupled with feature engineering for classification of phishing URLs using over 50 URL based and content based features and obtained classification accuracies greater than 97% . Similarly, a heuristic based framework was developed by Jadhav and Chandre [8] that employs analysis of features from multiple domains to classify phishing sites including URL analysis, email header , HTML contents and web page analysis along with recursive feature elimination resulting in a higher robustness in varying situations of phishing attacks and achieving accuracies of 96.7% and 96.4% respectively on URL datasets.

These heuristic based approaches require low training computation but lack rigidity. The pre determined and hand crafted rules may fail against new phishing strategies.

C. Machine learning - based approaches

The usage of ML techniques have received increasing attention for their ability to capture and understand complex patterns and generalize previously unobserved phishing attacks via URLs. They use features derived from URLs, website content, or domain attributes. More recently, the effectiveness of ML classifiers such as Random Forest, Support Vector Machines (SVM), and Logistic Regression for the task of detecting phishing links have been shown. Akhil Mittal et al.[9] suggests a ML-based phishing detection system which uses SVMs (support vector machines), decision tree and neural network combined with feature engineering that emphasizes adaptability and high accuracy.. A comparison of different ML classifiers on phishing website detection by Almujahid et al.[10] has also found that ensemble , tree-based classifiers and CNNs (convolutional neural network) are performing better than others consistently achieving accuracies above 98%.

Deep learning methods have shown promising improvements. Md Sultanul Islam Ovi et al.[11] shows that an ensemble model combining Random forest , Catboost, XGBoost and gradient boost performs better than traditional methods on complex, large-scale datasets for phishing websites. This fusion of various ML classifiers into a hybrid model have also been employed to yield state-of-the-art results achieving accuracy of 99.05%. Rumman Firdos et al.[12] suggested a combined approach to integrate heuristic techniques, ML, and threat intelligence into a multi-layer AI model for real-world application. Brij.B Gupta et al.[13] suggested using CNN for capturing structural pattern and BERT transformer for capturing contextual pattern for phishing link detection but suffers from high computational cost of CNN and BERT transformer.

Current research trends have been focusing on hybrid models to combine above methods to obtain effective and scalable phishing detection systems.

METHODOLOGY

The proposed methodology is divided into seven phases : -

1. Data acquisition.
2. Data pre processing.
3. Data splitting into training and testing sets.
4. Model building and initializing.
5. Model training.
6. Training results.
7. Model evaluation / testing.

A. Data Acquisition

We combined two different datasets from kaggle into a single bigger dataset for. One of the dataset is the PhiUSIIL phishing URL dataset from kaggle and is available publicly under CC BY-NC-SA 4.0 license. This dataset has 2,35,795. Other dataset is also taken from kaggle and is available to the public under Database: Open Database, Contents license. This one has 5,49,346 URL entries. Combining both dataset we have 7,85,141. Out of these 5,27,774 URLs are legitimate and 2,57,367 URLs are phished.

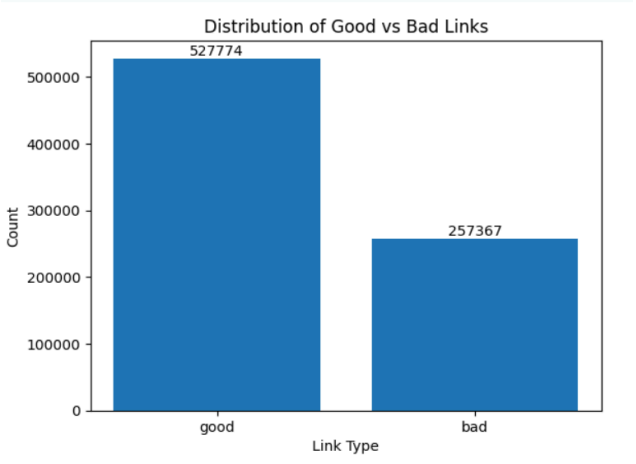

Fig 1. Class distribution of the dataset

B. Data pre processing

The preprocessing pipeline aims to convert the raw URL data into a textual representation that is structured and enriched with features of security importance that are learnable by the transformer-based models. The pipeline is composed of several ordered stages namely normalization, decomposition, feature enrichment and tokenization.

1. *Integrating and Cleaning Datasets*: The dataset is preprocessed to remove missing values. Labels of both datasets are united and converted into a binary classification system (good, bad) and then label encoded to represent numerical values. Random shuffling of combined dataset to prevent order bias.

2. *URL Normalization*: Canonical forms of each URL are produced -
   - Converting all characters to lowercase.
   - Removing white space at beginning and end.
   - Substituting 'http://' or 'https://' with the token 'http'.
   - Separating 'www' into distinct token 'www'.

3. *Robust URL Parsing*: Adding a default scheme 'http://' to URL and using a utility function to parse. If failing in parsing, the control continues to the next URL without causing a system crash.

4. *URL Decomposition*: The normalized URL is then broken down into several elements-
   - Domain: Replaced '.' with a spaced token ' . ' to show domain hierarchy.
   - Path: Replaced '/' with another spaced tokens ' / ' to preserve the directory.
   - Query Parameters: Special characters like '&', '=' are too represented as tokens ' & ' and ' = '.

5. *Security-Oriented Feature Augmentation*: Hand-crafted security-related tokens are added at the end of the URL. They are helpful in identifying common patterns in malicious URLs. These tokens represents -
   - Keywords indicating maliciousness (login, verify, update, etc) present in the URL.
   - Presence of long number sequence (≥3 digits), often used in obfuscation.
   - Presence of many hyphens (> 2).
   - URL too long (> 75 characters).
   - Presence of '@'.
   - Many subdomains (> 3 dots).

   Each detected feature is added at the end as a textual token (e.g., *long_URL*, *many_hyphens*), thereby enriching the semantic representation.

6. *Final Text Representation*: The raw normalized URL, decomposed URL structure and security tokens are concatenated to form the final text sequence for each URL.

7. *Custom dataset and tokenization* : For training , to handle the data in an efficient manner, a custom dataset class is created using the PyTorch Dataset interface. This class converts the pre-processed URLs into an input suitable for the transformer. For each sample, it uses the DistilBERT tokenizer to tokenize the pre-processed URL into input token IDs vector and corresponding attention masks with a fixed length padding and truncation to keep vector size the same for all URLs (length 64). Labels for each sample are also converted to tensor. The class returns input IDs, attention masks and labels.

C. Data splitting into training and testing set

The entire dataset is splitted into training and testing sets using python's built in train_test_split utility. The training set comprises 80% of the entire dataset. The testing set comprises the remaining 20% of the entire dataset. Training loaders and testing loaders are then created with batch size of 512 from training and testing set respectively.

D. Model building and initializing

The model consists of DistilBERT transformer encoder for contextual embeddings feature extraction , multi layer perceptron (MLP) for non linear transformation followed by a late feature level fusion via concatenation and a final classification layer. MLP (multi layer perceptron) has a hidden layer of 256 neurons and an output layer of 128 neurons with a dropout of 0.5 to prevent overfitting. For ease of exposition, all tensor dimensions are described with respect to a single input instance (i.e., batch size = 1). In practice, the model is trained using mini-batches of size 512, and all operations are applied in a batched manner. The transformer is fine-tuned end-to-end and used to generate contextual embeddings. It takes a token id vector and attention mask both of shape [1x64] as input. The DistilBERT transformer has 6 transformer layers and uses multi head self attention to capture global dependencies within URL tokens. It outputs a feature tensor of shape [1x64x768]. CLS token extraction is performed to get the output tensor from the transformer having shape [1x768]. This tensor acts as a contextual embedding feature representation and is fed as input to a multi-layer perceptron. The hidden layer of the MLP performs the operation:

$$h_1 = \text{ReLU}\,(V_{\text{DistilBERT}} W_1 + b_1)$$

Where $V_{\text{DistilBERT}}$ is a contextual embedding feature representation of shape [1x768] and W_1 is a weight matrix for the hidden layer of shape [768x256] and b_1 is a bias vector of shape [1x256]. ReLU activation is also applied on the hidden layer and output h_1 is taken from this layer and

has a shape of [1x256]. The output layer of the MLP performs the same operation as the hidden layer but on the tensor h_1 and outputs the tensor of shape [1x128], say V_{MLP}. The transformer output representation $V_{DistilBERT}$ and MLP output representation V_{MLP} are concatenated in a late feature level fusion resulting in a vector of shape [1x896]. This is then passed through a dropout layer (0.3) and to the final binary classification layer to get the final label. The model's architecture is shown below.

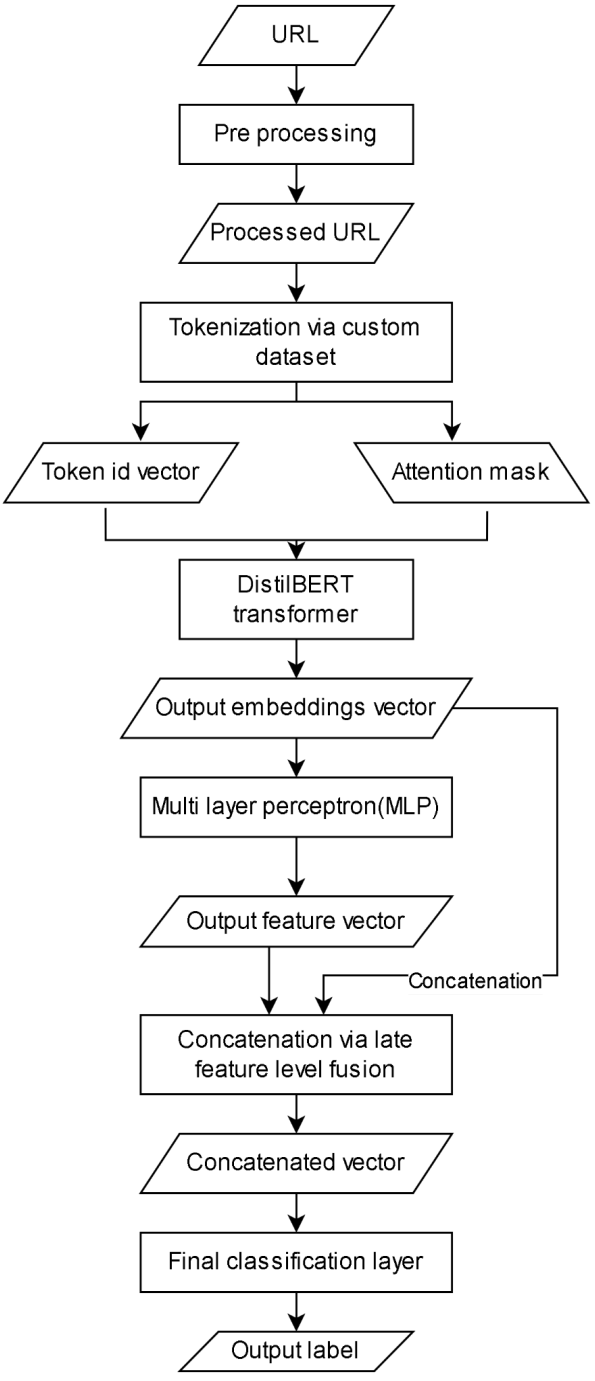

Fig 2. System architecture

E. Model training

The model is trained on the training dataset for three epochs on google colab's T4 tesla GPU which took about 105 minutes , averaging 35 minutes per epoch. Loss is calculated and then backpropagated after each epoch. AdamW optimizer is used with a learning rate of 5e-5.

F. Training results

The training loss is computed for each epoch during the training loop. Loss after epoch 1 , epoch 2 and epoch 3 are 0.0568 , 0.0228 and 0.0122 respectively.

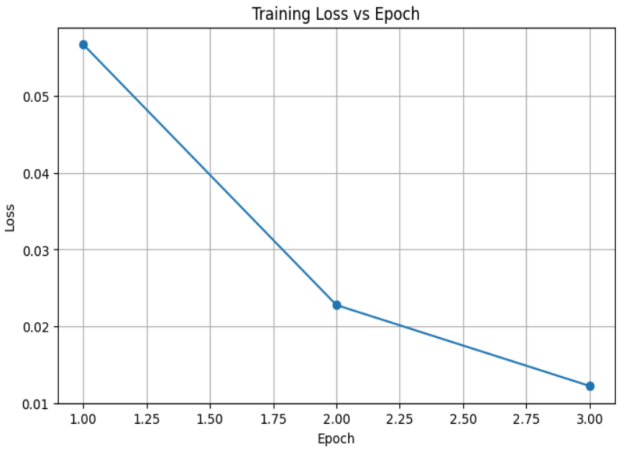

Fig 3. Training loss vs epoch

G. Model evaluation and results

The testing is carried out on the testing dataset which is 20% of the entire dataset. It consists of 1,57,029 URLs. The model achieved an accuracy of approximately 99.25% with recall , precision and f1 score of 0.9925 , 0.9925 and 0.9925 respectively.

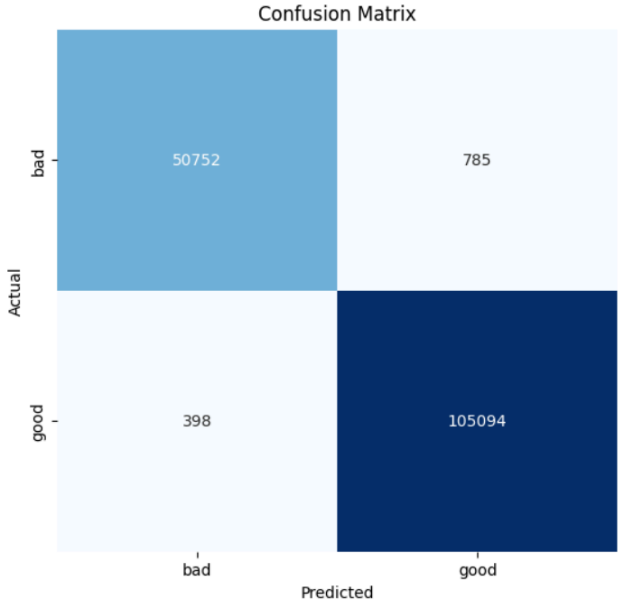

Fig 4. Confusion matrix

```
--- Overall Evaluation Metrics ---
Accuracy: 99.2466
Precision: 0.9925
Recall: 0.9925
F1-score: 0.9925

Classification Report:
              precision    recall  f1-score   support

         bad       0.99      0.98      0.99     51537
        good       0.99      1.00      0.99    105492

    accuracy                           0.99    157029
   macro avg       0.99      0.99      0.99    157029
weighted avg       0.99      0.99      0.99    157029
```

Fig 5. Classification report of proposed model

Models	Accuracy	Recall	Precision	F1-score
Proposed model	*99.246%*	*0.9925*	*0.9925*	*0.9925*
CNN	98.70%	0.986	0.986	0.985
Random forest	96.16%	0.961	0.962	0.961
Decision tree	91.64%	0.916	0.916	0.914
Logistic regression	94.90%	0.949	0.949	0.947

Fig 6. Comparison with other ML models.

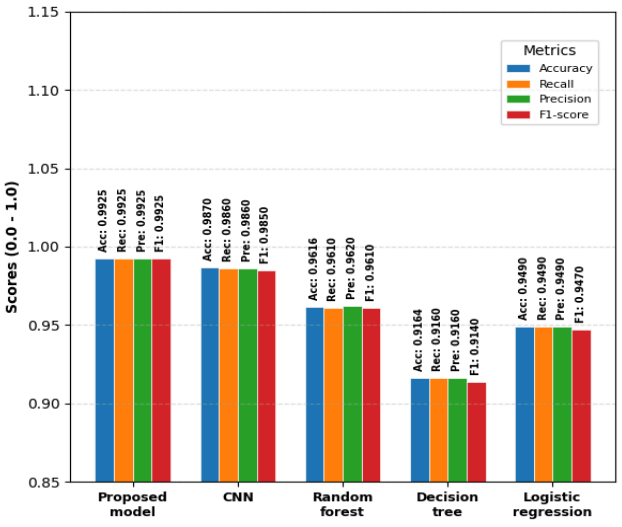

Fig 7. Comparison graph between proposed mode and other models

EXPERIMENTAL SETUP

The experiment is performed on google's colab platform. The libraries used are :

- *Transformer* : For tokenization and DistilBERT transformer encoder model.
- *Pytorch* : Deep learning framework.
- *Scikit - learn* : For data preprocessing and splitting.
- *Numpy and pandas* : For numerical data manipulation.
- *Matplotlib and saeborn* : For data visualization.
- *XGBoost* : Baseline ML model.

The setup also uses Nvidia's Tesla T4 GPU for training the model. The VRAM used in the T4 GPU has a capacity of 16 GB. To maximize the VRAM usage and efficiency, batch sizes of 512 are used during training and testing. VRAM usage at this batch size is consistently high at approximately 13 GB. Larger batch sizes often crash the entire session meanwhile , smaller batch sizes failed to maximize VRAM usage and efficiency.Hence , batch size of 512 is found to be optimal for this setup.

CONCLUSION AND FUTURE SCOPE

The proposed model achieved significantly higher accuracy , precision , recall and f1 score than traditional ML models and existing list based and heuristic based methods. The tests were conducted on a big dataset with a diverse set of URLs which shows great generalizing capability of the model. This experiment also shows that most transformer based models generally outperforms other models but their computational cost is their limitation. Combining light weight multi layer perceptron (MLP) with a light weight transformer , we can significantly bring the computational cost down while maintaining high accuracy.

For future works , we might look to bring computational cost even more down than what we achieved with our proposed work. The proposed work can also be deployed as a standalone application with UI/UX for phishing detection. We will also continue to look for ways to integrate the proposed approach into other fields like quantum cryptography in which we can investigate hybrid frameworks to explore transformer based phishing detection with quantum resistant protocols for end-to-end security.

REFERENCES

1) Jain, Ankit & Gupta, Brij B. (2017). Phishing Detection: Analysis of Visual Similarity Based Approaches. Security and Communication Networks. 2017. 1-20. 10.1155/2017/5421046.
2) Verizon, "Data Breach Investigations Report," 2025.
3) Aburrous, Maher & Hossain, Mohammed & Dahal, Keshav & Thabtah, Fadi. (2010). Intelligent phishing detection system for e-banking using fuzzy data mining. Expert

Systems with Applications. 37. 7913-7921. 10.1016/j.eswa.2010.04.044.
4) Garera, Sujata & Provos, Niels & Chew, Monica & Rubin, Aviel. (2007). A framework for detection and measurement of phishing attacks. WORM'07 - Proceedings of the 2007 ACM Workshop on Recurring Malcode. 10.1145/1314389.1314391.
5) Sanh, Victor & Debut, Lysandre & Chaumond, Julien & Wolf, Thomas. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. 10.48550/arXiv.1910.01108.
6) Kyaw PH, Gutierrez J, Ghobakhlou A. A Systematic Review of Deep Learning Techniques for Phishing Email Detection. *Electronics*. 2024; 13(19):3823. https://doi.org/10.3390/electronics13193823.
7) Jayaprakash R, Natarajan K, Daniel JA, Chinnappan CV, Giri J, Qin H and Mallik S (2024) Heuristic machine learning approaches for identifying phishing threats across web and email platforms. Front. Artif. Intell. 7:1414122. doi: 10.3389/frai.2024.1414122
8) A. Jadhav and P. Chandre, "A Hybrid Heuristic-Machine Learning Framework for Phishing Detection Using Multi-Domain Feature Analysis", *Eng. Technol. Appl. Sci. Res.*, vol. 15, no. 5, pp. 27219–27226, Oct. 2025.
9) Akhil Mittal. (2024). Machine Learning-Based Phishing Detection: Improving Accuracy and Adaptability. *International Journal of Intelligent Systems and Applications in Engineering*, *12*(22s), 587–595. Retrieved from https://ijisae.org/index.php/IJISAE/article/view/6524.
10) Almujahid NF, Haq MA, Alshehri M. Comparative evaluation of machine learning algorithms for phishing site detection. PeerJ Comput Sci. 2024 Jun 24;10:e2131. doi: 10.7717/peerj-cs.2131. PMID: 38983211; PMCID: PMC11232597.
11) Ovi, Md Sultanul Islam, Md Hasibur Rahman, and Mohammad Arif Hossain. "Phishguard: A multi-layered ensemble model for optimal phishing website detection." *2024 6th International Conference on Sustainable Technologies for Industry 5.0 (STI)*. IEEE, 2024.
12) Firdos, R., & Dangi, A. (2026). SecureScan: An AI-Driven Multi-Layer Framework for Malware and Phishing Detection Using Logistic Regression and Threat Intelligence Integration. *arXiv preprint arXiv:2602.10750*.
13) Gupta, B.B., Gaurav, A., Arya, V., Attar, R.W., Bansal, S. et al. (2024). Advanced BERT and CNN-Based Computational Model for Phishing Detection in Enterprise Systems. *Computer Modeling in Engineering & Sciences*, *141(3)*, 2165–2183. https://doi.org/10.32604/cmes.2024.056473.
14) Shanmugam, Kavya & Sumathi, D.. (2024). Staying ahead of phishers: a review of recent advances and emerging methodologies in phishing detection. Artificial Intelligence Review. 58. 10.1007/s10462-024-11055-z..
15) Wilk-Jakubowski, J.L.; Pawlik, L.; Wilk-Jakubowski, G.; Sikora, A. Machine Learning and Neural Networks for Phishing Detection: A Systematic Review (2017–2024). *Electronics* 2025, *14*, 3744. https://doi.org/10.3390/electronics14183744